

1.1.1. Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:

m is the mass of the object (in kilograms).

v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases +

```
calculate... Submit
1 #Type Content here...
2 mass = float(input())
3 velocity = float(input())
4 momentum = mass * velocity
5 print(f"{momentum:.2f}kgm/s")

Terminal Test cases
```

1.1.1. Calculate Momentum

00:31

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:

m is the mass of the object (in kilograms).

v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases

+

calculate...

Submit

```
1 #Type Content here...
2 mass = float(input())
3 velocity = float(input())
4 momentum = mass * velocity
5 print(f"{momentum:.2f}kgm/s")
```

Average time

0.013 s

13.00 ms



Maximum time

0.014 s

14.00 ms



2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 14 ms

Debug

Test cases

Expected output

5.0

10.0

50.00kgm/s

Actual output

5.0

10.0

50.00kgm/s

Test case 2 11 ms

Terminal

Test cases



1.1.2. Conditional Calculation Based on the Number of Digits

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:

$1 \leq n \leq 999$

Input Format:

- The input consists of a single integer n .

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Sample Test Cases

condition...

Submit

Submit

```
1 #write your code here...
2 import math
3
4 def solve():
5     try:
6         # Read the input integer
7         line = input().strip()
8         if not line:
9             return
10        n = int(line)
11    
```

Average time
0.008 s
7.71 ms

Maximum time
0.010 s
10.00 ms

4 out of 4 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 8 ms Debug

Expected output
9
81

Actual output
9
81

Test case 2 10 ms

Test case 3 8 ms

Terminal Test cases



1.1.2. Conditional Calculation Based on the Number of Digits

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:

$1 \leq n \leq 999$

Input Format:

- The input consists of a single integer n .

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Sample Test Cases

condition...

1

#write your code here...

2

import math

3

4

def solve():

5

try:

6

Read the input integer

7

line = input().strip()

8

if not line:

9

return

10

n = int(line)

11

Average time

0.008 s

7.71 ms

Maximum time

0.010 s

10.00 ms

4 out of 4 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1

8 ms

Debug

Expected output

Actual output

9

9

81

81

Test case 2

10 ms

Test case 3

8 ms

Terminal

Test cases



1.1.3. Days between Two Dates

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format *YYYY – MM – DD*.
- The second line contains the second date in the format *YYYY – MM – DD*.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

noOfDay...

Submit

```
1 from datetime import date
2 from datetime import datetime
3
4 def calculate_days():
5     # Read the two input dates as strings
6     date_str1 = input().strip()
7     date_str2 = input().strip()
8
9     # Define the format specified in the problem: YYYY-MM-DD
10    date_format = "%Y-%m-%d"
11
```

Average time
0.015 s
14.50 ms

Maximum time
0.036 s
36.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 36 ms Debug

Expected output	Actual output
2014-07-02	2014-07-02
2014-11-02	2014-11-02
123	123

Test case 2 8 ms

Terminal

Test cases



1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases

reverseN...

```
1 #write your code here...
2 x = int(input())
3 sum = 0
4 while(x>0):
5     rem = x%10
6     sum = sum*10+rem
7     x = x//10
8 print(sum)
```

Average time

0.004 s

3.80 ms

Maximum time

0.005 s

5.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1 5 ms

Debug

Expected output

5367

7635

Actual output

5367

7635

✓ Test case 2 3 ms

Terminal

Test cases



1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases



Explorer multiplica... Submit

```
1 #write your code here...
2
3 # Read the integer input
4 n = int(input())
5
6 # Loop from 1 to 10 inclusive
7 for i in range(1, 11):
8     # Calculate the result
9     result = n * i
10    # Print using the specific format required: <number> x <multiplier> =
    <result>
```

Average time: 0.005 s (4.75 ms) Maximum time: 0.007 s (7.00 ms)

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 7 ms Debug

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40

Terminal Test cases



1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses. The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

passorFa...

Submit

```
1 # write your code here
2 n = int(input())
3
4 # Read marks as a list of integers
5 marks = list(map(int, input().split()))
6
7 # Check if student failed any individual course (mark < 40)
8 failed = any(m < 40 for m in marks)
9
10 if failed:
11     print("Fail")
```

Average time
0.007 s
7.30 ms

Maximum time
0.010 s
10.00 ms

5 out of 5 shown test case(s) passed
5 out of 5 hidden test case(s) passed

Test case 1 8 ms Debug

Expected output
4
55 65 78 89
Aggregate Percentage: 71.75
Grade: First Division

Actual output
4
55 65 78 89
Aggregate Percentage: 71.75
Grade: First Division

Test case 2 6 ms

Terminal

Test cases



1.2.2. Fibonacci Series

05.24

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases +

fibonacci...

```
1 '''def fibonacci(n):
2
3     # write the code..
4
5     return fibonacci(...)
6
7
8 n = int(input())
9 for i in range(1, n + 1):
10     print(fibonacci(i), end=" ")
11 '''
```

Average time

0.019 s

19.00 ms

Maximum time

0.056 s

56.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 8 ms

Debug

Expected output

5

0 1 1 2 3

Actual output

5

0 1 1 2 3

Test case 2 4 ms

Terminal

Test cases



1.2.3. Pattern - 1

00:25

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

+

rightangl...

Submit

```
1 #Type Content here...
2 rows = int(input())
3
4 for i in range(1, rows + 1):
5     # Each row contains 'i' asterisks separated by spaces
6     print("* " * i)
```

Average time

0.004 s

3.83 ms



Maximum time

0.005 s

5.00 ms



2 out of 2 shown test case(s) passed

4 out of 4 hidden test case(s) passed

Test case 1 5 ms

Debug

≡

📄

Expected output

5

* *
* * *
* * * *
* * * * *
* * * * *
* * * * *

Actual output

5

* *
* * *
* * * *
* * * * *
* * * * *
* * * * *

Terminal

Test cases



1.2.4. Pattern - 2

00:24

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases

+

Explorer

numberP...

Submit

```
1 # Type Content here...
2 def print_number_triangle(rows):
3     for i in range(1, rows + 1):
4         for j in range(1, i + 1):
5             print(j, end=" ")
6         print()
7
8 # Input: Number of rows in the pattern
9 rows = int(input())
10
11 # output: Right-angled triangle pattern of numbers
```

Average time

0.004 s

4.25 ms

Maximum time

0.005 s

5.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 4 ms

Debug

Test cases

Expected output

5

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Actual output

5

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Terminal

Test cases

